Segurança da Informação– GBC083

Prof. Rodrigo Sanches Miani – FACOM/UFU

Tópicos

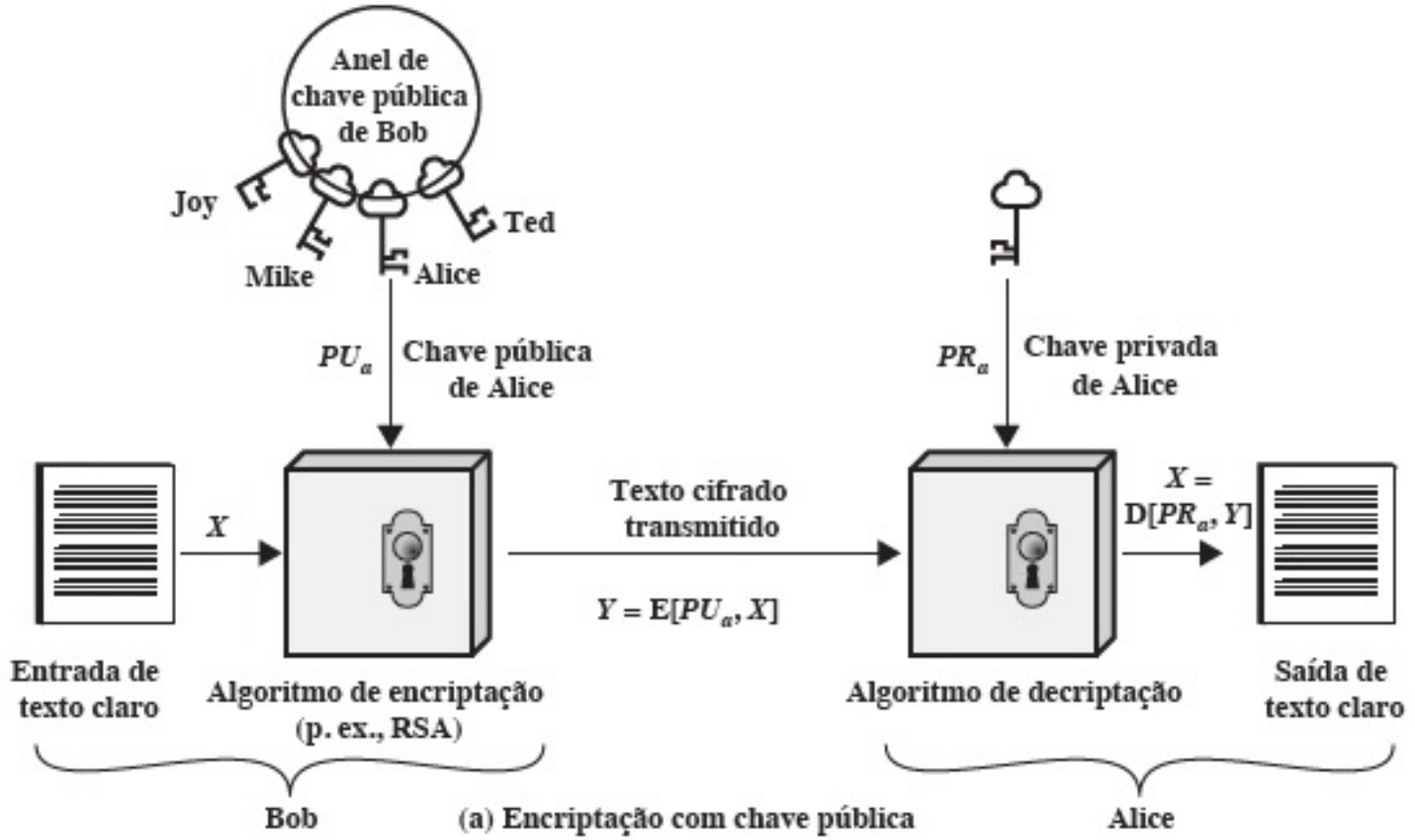
1. Resumo da semana 9
2. Direcionamentos para a semana 10
3. Dúvidas??



Resumo da semana 9

Segurança da Informação– GBC083

Elementos


Anel de chave pública de Bob
Joy
Ted
Mike
Alice
PU_a
Chave pública de Alice
PR_a
Chave privada de Alice
X
Texto cifrado transmitido
$Y = E[PU_a, X]$
$X = D[PR_a, Y]$
Entrada de texto claro
Algoritmo de encriptação (p. ex., RSA)
Algoritmo de decriptação
Saída de texto claro
Bob
(a) Encriptação com chave pública
Alice

Requisitos

1. É computacionalmente fácil* para uma entidade B gerar um par (PU_b PR_b);

2. É computacionalmente fácil* que um emissor A, conhecendo a chave pública e a mensagem a ser cifrada, *M*, gere o texto cifrado: $C = E(PU_b, M)$;

3. É computacionalmente fácil* que o receptor B decifre o texto cifrado C usando a sua chave privada;

* Fácil significa um problema que pode ser resolvido em tempo polinomial como função do tamanho da entrada.



Requisitos

4. É computacionalmente inviável* que um atacante, conhecendo a chave pública, PU_b, determine a chave privada PR_b;

5. É computacionalmente inviável* que um atacante, conhecendo a chave pública, PU_b e um texto cifrado C recupere a mensagem original M.

* Um problema é inviável se o esforço para solucioná-lo aumentar mais rapidamente do que o tempo polinomial em função do tamanho da entrada



Requisitos

A ideia é construir uma função que tenha a seguinte propriedade:

- $C = E(PU_b, M)$ – fácil de calcular
- $M = D(PR_b, C)$ – fácil de calcular, se eu souber PR_b
  - Note que D é a inversa de E. A notação poderia ser E=f e $D = f^{-1}$

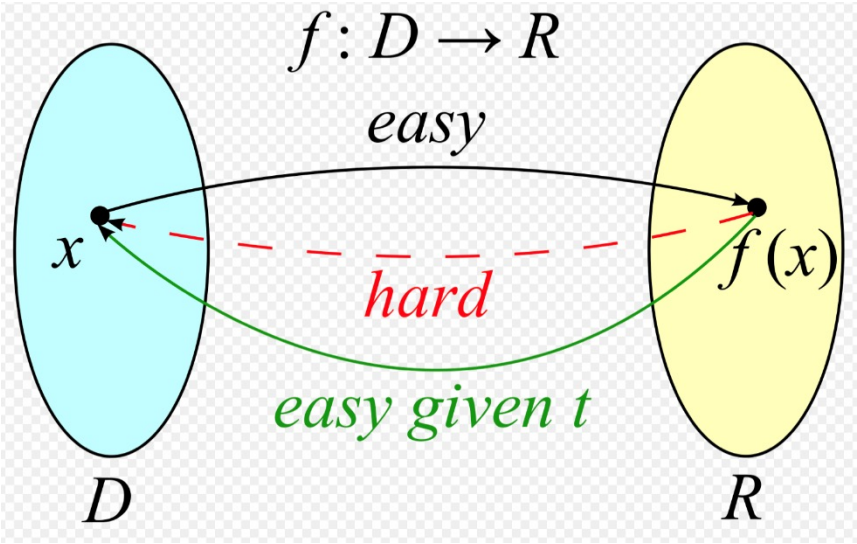
- $M = D(?, C)$ – inviável se eu não souber PR_b



Requisitos

- A função f deve ser uma *trapdoor function*;

- Fácil de se calcular em uma direção e inviável na outra, a menos que certa informação adicional seja conhecida.





Aplicações

Três aplicações para sistemas de criptografia assimétricos:

1. Cifrar/decifrar – confidencialidade;

2. Assinatura digital – não repúdio;

3. Troca de chaves – dois lados cooperam para trocar uma chave de sessão.
   - Vimos que o Diffie-Hellman proporciona isso. É possível estabelecer um modelo de troca de chaves usando somente as premissas de criptografia assimétrica?



Criptografia – Modelo híbrido

- Criptografia simétrica:
  - Vantagem: boa performance computacional;
  - Desvantagem: troca de chaves;
- Criptografia assimétrica:
  - Vantagem: permite a troca segura de chaves;
  - Desvantagem: desempenho computacional ruim;
- Porque não unir os dois modelos?



Direcionamentos para a semana 9

Segurança da Informação– GBC083

Direcionamentos para a semana 9

- Um conjunto de slides
  1. Tópico 10 – Criptografia assimétrica

- Uma videoaula:
  1. Tópico 10 – RSA (1 hora)



Direcionamentos para a semana 8

- QP7 será somente na semana que vem! Amanhã nós não teremos aula!
- TP4 até 21/02! Como estão as coisas?



Tópicos da aula – RSA (videoaula)

1. Ideia
2. Descrição do algoritmo
3. Por que funciona?



Resumo da aula...

- RSA
  - Algoritmo que implementa os requisitos de criptografia de chave pública;
  - Usa aritmética modular, primalidade e outros argumentos da **teoria de números**;
  - A descrição é razoavelmente simples mas a escolha dos parâmetros pode dificultar a sua implementação (e levar a escolhas muito ruins!);
  - A segurança do algoritmo está diretamente relacionada ao problema de fatoração de números inteiros.



Histórico

- Em 1978, três professores do MIT (Ron Rivest, Adi Shamir e Len Adleman) publicaram uma proposta de um algoritmo de criptografia de chave pública;

- O algoritmo, teoricamente, satisfazia as cinco condições de funcionamento de criptografia de chave pública.



Ideia

- A função f, *trapdoor*, escolhida foi baseada em uma série de argumentos de uma área da matemática conhecida como *teoria dos números*;

- Tal área estuda propriedades de números inteiros como divisibilidade, primalidade e fatoração.



Ideia

Relembrando... A função f, *trapdoor:*

- C = E(PU_b, M) – fácil de calcular
- M = D(PR_b, C) – fácil de calcular, se eu souber PR_b
  - Note que D é a inversa de E. A notação poderia ser E=f e D = f^{-1}

A cifragem e decifragem no RSA devem ser feitas da seguinte forma:

- C = M^e mod n
- M = C^d mod n
- Ou seja, e seria a chave pública e *d* a chave privada;
- O grande "truque" é definir uma função tal que i) e,*d* sejam inversos e ii) dado somente e, é inviável para um atacante descobrir *d*.



Ideia

2^6 mod 10 = 4 (resto da divisão de 64 por 10 é 4)

Se M = 2 e C = 4 será que não existe algum número tal que:

4^x mod 10 = 2?

Ou seja, será que eu consigo recuperar M à partir do texto cifrado? Quem deveria ser x?



Características básicas do RSA

- RSA é uma cifra de bloco em que o texto claro e o cifrado são inteiros entre 0 e n-1 para algum n;
- Um tamanho típico para n é 1024 bits, ou 309 dígitos decimais;
  - O NIST já recomenda que o tamanho de n seja 2048 bits.



Descrição do algoritmo

- O RSA pode ser descrito em três etapas:
  1. Criação das chaves;
  2. Cifrar;
  3. Decifrar.



Descrição do algoritmo – Geração de chaves

Geração de chave por Alice

Selecione p, q	p e q são primos, $p \neq q$
Calcule $n = p \times q$	
Calcule $\phi(n) = (p - 1)(q - 1)$	
Selecione o inteiro e	$\text{mdc}(\phi(n), e) = 1; 1 < e < \phi(n)$
Calcule d	$d \equiv e^{-1} \pmod{\phi(n)}$
Chave pública	$PU = \{e, n\}$
Chave privada	$PR = \{d, n\}$



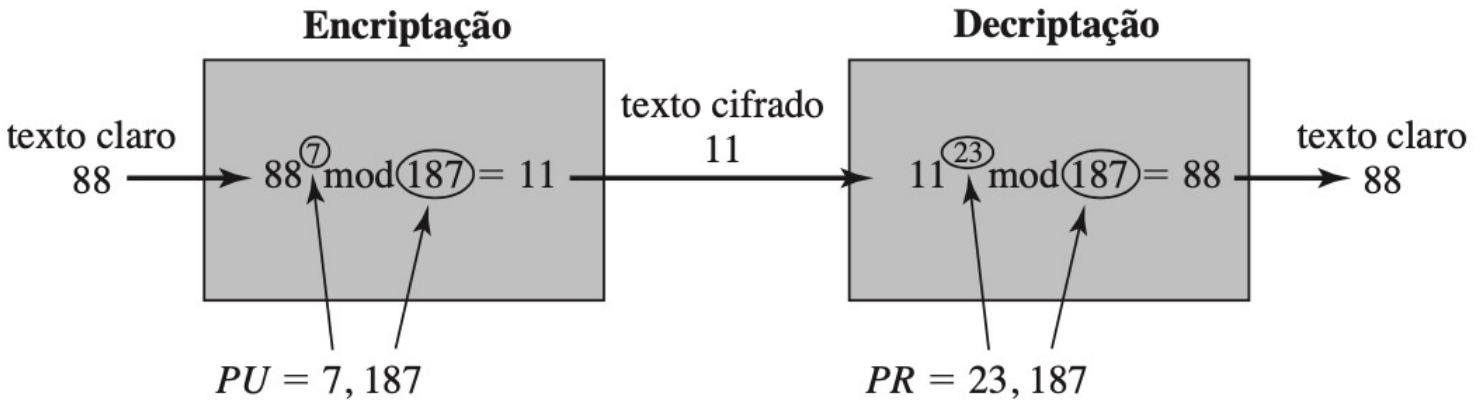
Descrição do algoritmo – Cifrar/Decifrar

Encriptação por Bob com chave pública de Alice	
Texto claro:	$M < n$
Texto cifrado:	$C = M^e \bmod n$

Decriptação por Alice com a chave privada de Alice	
Texto cifrado:	C
Texto claro:	$M = C^d \bmod n$



RSA - Exemplo


Encriptação
Decriptação
texto claro
88
$88^{7} \bmod 187 = 11$
texto cifrado
11
$11^{23} \bmod 187 = 88$
texto claro
88
$PU = 7, 187$
$PR = 23, 187$

Segurança do RSA



Segurança do RSA

- O par (*n*,e) é acessível a qualquer usuário ou seja, o RSA só será seguro se for difícil calcular *d* quando apenas (*n*,e) são conhecidos;

- Para encontrar d, precisamos aplicar o *algoritmo euclidiano estendido* a $\phi(n)$ e e, porém para encontrar $\phi(n)$ precisamos conhecer *p* e *q*, ou seja, só podemos quebrar o RSA se soubermos fatorar *n*.
  - O que acontece se n for muito grande?
  - Nenhum algoritmo atualmente disponível/publicado consegue fatorar números em tempo polinomial (acredita-se que esse seja um problema NP-completo);
  - Um número de 768 bits formado por dois fatores primos, foi fatorado com sucesso no fim de 2009 – pesquisadores levaram 2 anos para se fazer isso!



Escolha dos parâmetros

Como escolher p e q?

Existe alguma técnica para "gerar" números primos?



Escolha dos parâmetros

Como escolher p e q?

Existe alguma técnica eficiente para "gerar" números primos?

Resposta: não! A ideia é escolher um número ímpar e testar se ele é primo...

Existem diversos testes de primalidade que podem ser usados para escolher p e q.

A grande maioria dos testes implementados na prática são probabilísticos... Alguém sabe o porquê disso?



Até quando podemos confiar no RSA?

NÚMERO DE DÍGITOS DECIMAIS	NÚMERO DE BITS	DATA EM QUE FOI ALCANÇADO
100	332	abril de 1991
110	365	abril de 1992
120	398	junho de 1993
129	428	abril de 1994
130	431	abril de 1996
140	465	fevereiro de 1999
155	512	agosto de 1999
160	530	abril de 2003
174	576	dezembro de 2003
200	663	maio de 2005
193	640	novembro de 2005
232	768	dezembro de 2009



RSA na prática

- RSA é implementado em diversas bibliotecas criptográficas, SSL, TLS, OpenSSL, BouncyCastle, Web Crypto API e por aí vai...
  1. A escolha dos parâmetros sempre é um ponto crítico!
  2. Além disso, o RSA também costuma ser implementado "do zero" em serviços "em produção" – isso não é uma boa prática!

- Esses dois fatores levaram a uma grande discussão nos últimos anos sobre a adoção do RSA:
  - https://blog.trailofbits.com/2019/07/08/fuck-rsa/
  - https://securityboulevard.com/2018/12/why-tls-1-3-is-a-huge-improvement/
  - A última versão do TLS RETIROU o RSA do processo de troca de chaves...



Dúvidas??

Segurança da Informação– GBC083